Readings:

- Why Container Security Matters
- Large Scale Cluster Management at Google with Borg
- Running Containerized Microservices on AWS
- OWASP Top 10 2021
- Prisma Cloud Product Overview (EN/DE)

# Lecture 4: Container Security

Lucerne University of
Applied Sciences and Arts
HOCHSCHULE
LUZERN

# Overview

- What is a container?
- Why use?
- Container services and examples
- Security issues around containers
- Security solutions
  - Kubernetes

# What are containers?



- A logical packaging mechanism in which applications can be abstracted from the environment in which they actually run
- This decoupling allows container-based applications to be deployed easily and consistently regardless of whether target environment is a private data centre, the public cloud or even a laptop.
- Separation of focus areas:
  - Developers focus on application logic and dependencies
  - IT Ops teams focus on deployment and management without concern for specific application(s)



https://cloud.google.com/containers

# Benefits of containers (1/2)

- Consistent Environment
  - Containers give developers the ability to create predictable environments that are isolated from other applications.
  - Containers can also include software dependencies needed by the application, such as specific versions of programming language runtimes and other software libraries.
  - From developer's perspective, all this is guaranteed to be consistent no matter where the application is ultimately deployed.
  - Developers and IT Ops teams spend less time debugging and diagnosing differences in environments, and more time shipping new functionality for users. And it means fewer bugs since developers can now make assumptions in dev and test environments they can be sure will hold true in production.

# Benefits of containers (2/2)

- Run Anywhere

  Containers are able to run virtually anywhere, greatly easing development and deployment: on Linux, Windows, and Mac operating systems; on virtual machines or bare metal; on a developer's machine or in data centers on-premises; and of course, in the public cloud.

  The widespread popularity of the Docker image format for containers further helps with portability. Wherever you want to run your software, you can use containers.

- Isolation

  Containers virtualize CPU, memory, storage, and network resources at the OS-level, providing developers with a sandboxed view of the OS logically isolated from other applications.

# Reasons for popularity of containerization

- **Flexible**: Even the most complex applications can be containerized.
- **Lightweight**: Containers leverage and share the host kernel, making them much more efficient in terms of system resources than virtual machines.
- **Portable**: You can build locally, deploy to the cloud, and run anywhere.
- **Loosely coupled**: Containers are highly self sufficient and encapsulated, allowing you to replace or upgrade one without disrupting others.
- **Scalable**: You can increase & automatically distribute container replicas across a DC.
- **Secure**: Containers apply aggressive constraints and isolations to processes without any configuration required on the part of the user.



https://docs.docker.com/get-started/

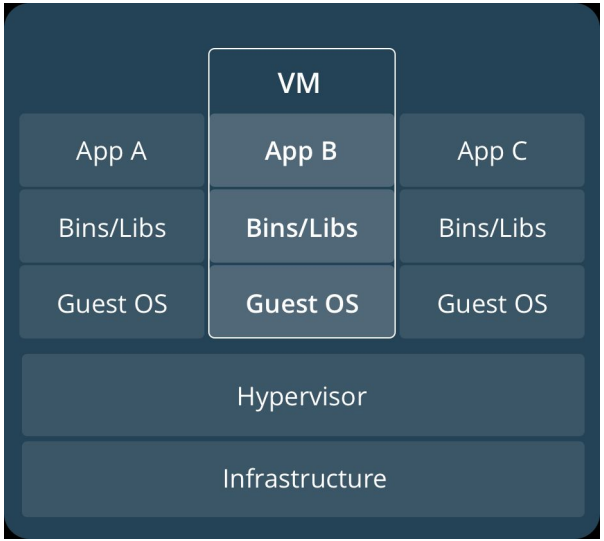
# Containers Virtualise at OS Level

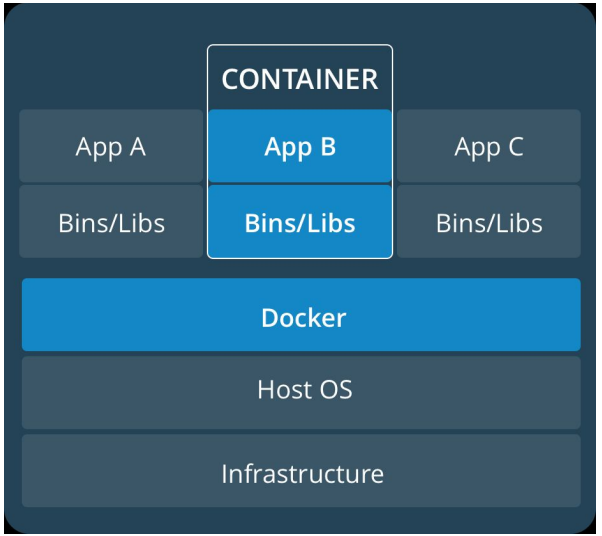
- Instead of virtualizing the hardware stack as with the virtual machines approach, containers virtualize at the operating system level, with multiple containers running directly on top of the OS kernel.
- This means that containers are far more lightweight: they share the OS kernel, start much faster, and use a fraction of the memory compared to booting an entire OS.
- There are many container formats available. Docker is a popular, open-source container format that is supported by many cloud service providers

# Containers and Virtual Machines (VMs)

By contrast, a **virtual machine** (VM) runs a full-blown "guest" operating system with *virtual* access to host resources through a hypervisor. In general, VMs incur a lot of overhead beyond what is being consumed by your application logic.

A container runs *natively* on Linux and shares the kernel of the host machine with other containers. It runs a discrete process, taking no more memory than any other executable, making it lightweight.






https://docs.docker.com/get-started/

# Docker Concepts

Docker is a platform for developers & sysadmins to **build, run, and share** applications with containers.

The use of containers to deploy applications is called *containerization*.

Containers are not new, but their use for easily deploying applications is.



https://docs.docker.com/get-started/

# Docker Images and Containers

- Fundamentally, a container is nothing but a running process, with some added encapsulation features applied to it in order to keep it isolated from the host and from other containers.
- One of the most important aspects of container isolation is that each container interacts with its own private filesystem; this filesystem is provided by a Docker **image**.
- An image includes everything needed to run an application - the code or binary, runtimes, dependencies, and any other filesystem objects required.



https://docs.docker.com/get-started/

# Sample Dockerfile

Writing a Dockerfile is the first step to containerizing an application. You can think of these Dockerfile commands as a step-by-step recipe on how to build up your image. The Dockerfile in the bulletin board app looks like this:

```
# Use the official image as a parent image.
FROM node:current-slim

# Set the working directory.
WORKDIR /usr/src/app

# Copy the file from your host to your current location.
COPY package.json .

# Run the command inside your image filesystem.
RUN npm install

# Add metadata to the image to describe which port the container is listening on at runtime.
EXPOSE 8080

# Run the specified command within the container.
CMD [ "npm", "start" ]

# Copy the rest of your app's source code from your host to your image filesystem.
COPY . .
```

https://docs.docker.com/get-started/part2/#sample-dockerfile

# Sandboxing

- Containers “silo” applications from each other - unless they are explicitly connected
- This means conflicting dependencies & resource contention are not issues
- Resource limits are set for each service
- Also presents an additional layer of security since applications are not running directly on the host OS

# Benefits Compared

| | Container Benefits | Virtual Machine Benefits |
|---|---|---|
| Consistent Runtime Environment | ✓ | ✓ |
| Application Sandboxing | ✓ | ✓ |
| Small Size on Disk | ✓ | |
| Low Overhead | ✓ | |

# From Monolithic to Service Based Architectures

## Monolithic to Service Based Architecture

Containers work best for service based architectures. Opposed to monolithic architectures, where every pieces of the application is intertwined — from IO to data processing to rendering — service based architectures separate these into separate components. Separation and division of labor allows your services to continue running even if others are failing, keeping your application as a whole more reliable.

Componentization also allows you to develop faster and more reliably; smaller codebases are easier to maintain and since the services are separate, it's easy to test specific inputs for outputs.

Containers are perfect for service based applications since you can health check each container, limit each service to specific resources and start and stop them independently of each other.

And since containers abstract the code away, containers allow you to treat separate services as black boxes, further decreasing the space a developer needs to be concerned with. When developers work on services that depends on another, they can easily start up a container for that specific service without having to waste time setting up the correct environment and troubleshooting beforehand.

# AWS Container Related Services

| Use case | AWS product |
| --- | --- |
| Store, encrypt, and manage container images | Amazon Elastic Container Registry (ECR) |
| Run containerized applications or build microservices | Amazon Elastic Container Serivce (ECS) |
| Manage containers with Kubernetes | Amazon Elastic Kubernetes Service (EKS) |
| Run containers without managing servers | AWS Fargate |
| Run containers with server-level control | Amazon Elastic Compute Cloud (EC2) |
| Containerize and migrate existing applications | AWS App2Container |
| Quickly launch and manage containerized applications | AWS Copilot |

# Quiz 7

- What is the difference between a VM and a Container?
- Provide some benefits of using Containers



# Production-Grade Container Orchestration: Kubernetes

- Kubernetes (open source project Kubernetes) or “k8s” (developed from the Google internal cluster management system Borg) provides automated container orchestration -- management machines and services-- improving reliability and reducing the time and resources which need to be spent on DevOps

# Production-Grade Container Orchestration: Kubernetes

- Kubernetes makes everything associated with deploying and managing applications easier.
  - Kubernetes automates rollouts and rollbacks, monitoring the health of services to prevent bad rollouts before things go bad.
  - Continuously runs health checks against your services, restarting containers that fail or have stalled, and only advertising services to clients when it has confirmed they've started up successfully.
  - Kubernetes will automatically scale your services up or down based off of utilization, ensuring you're only running what you need, when you need it.

# What’s in a Name?

- The name **Kubernetes** originates from Greek, **meaning** helmsman or pilot.



# Kubernetes vs Docker

- While the idea of isolating environments is not new and there are other types of containerization software, **Docker** has grown to be the **default container format** in recent years.
- Docker features the **Docker Engine**, which is a **runtime environment**.
  - allows you to build and run containers on any development machine; then store or share container images through a **container registry** like **Docker Hub** or **Azure Container Registry**.
- As applications grow to span multiple containers deployed across multiple servers, operating them becomes more complex. While Docker provides an open standard for packaging and distributing containerized apps, the potential complexities can add up fast.
  - How do you coordinate and schedule many containers? How do all the different containers in your app talk to each other? How do you scale many container instances? This is where Kubernetes can help.

# Kubernetes and Container Orchestration

- **Kubernetes** is **open-source orchestration software** that provides an API to control how and where those containers will run.
- It allows you to run your Docker containers and workloads and helps you to tackle some of the operating complexities when moving to scale multiple containers, deployed across multiple servers.
- **What is Kubernetes?**
  - Kubernetes lets you **orchestrate** a **cluster of virtual machines** and **schedule containers to run on those virtual machines** based on their available compute resources and the resource requirements of each container.
  - Containers are grouped into pods, the basic operational unit for Kubernetes.
  - Containers and pods can be scaled to your desired state and you're able to manage their lifecycle to keep your apps up and running.

# Docker Swarm

- **Docker**
  - **default container format**
  - **Docker Engine runtime environment**
  - **Docker Hub container registry**
  - **Docker Swarm.**
    - Docker Swarm provides a simple, straightforward way to orchestrate containers, often used in situations where companies feel their needs are not suitably complex to warrant using Kubernetes.
    - Offered as an alternative orchestration type with Mirantis Kubernetes Engine (MKE). Users can access the Mirantis Kubernetes Engine webUI to switch nodes to Swarm or 'mixed' (i.e., Kubernetes+Swarm) mode at will.
    - Open source Docker Engines can also be combined in a swarm, using CLI commands.
    - Swarm Services: application components that work together to create a full application. This may include the application itself, any external components it needs such as databases, and network and storage definitions.

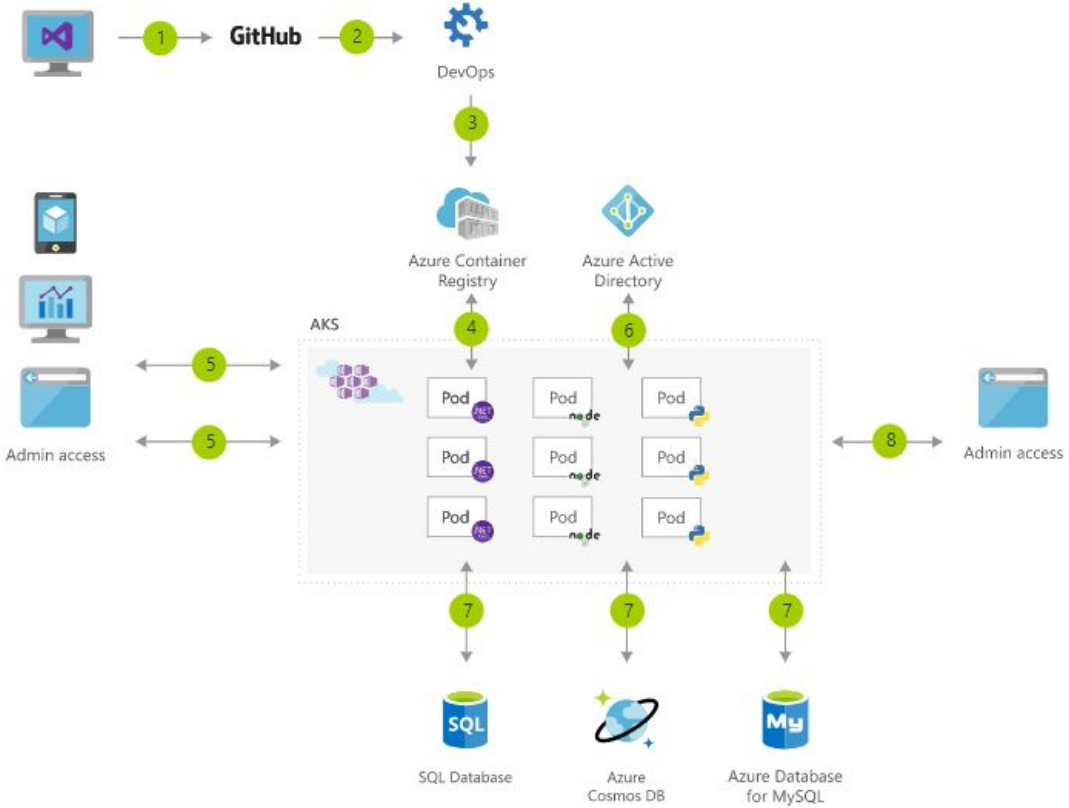
# Docker Swarm vs Kubernetes

- Each has its advantages
- Swarm gained a lot of traction to start because it is part of Docker itself, (developers don't need to add anything else; simpler to deploy and use than Kubernetes).
- Kubernetes, however, has long-since surpassed Swarm in usage, and has its own environments and adherents.
- That doesn't however, mean there's a clear answer as to which is "better". Many factors determine which is better for each deployment, such as existing environment, target environment, application complexity, and so on.
- There are utilities that translate Swarm "stacks" into Kubernetes definitions to enable moving from one to other.



https://www.mirantis.com/cloud-native-concepts/understanding-docker-swarm/what-is-docker-swarm/#section-3

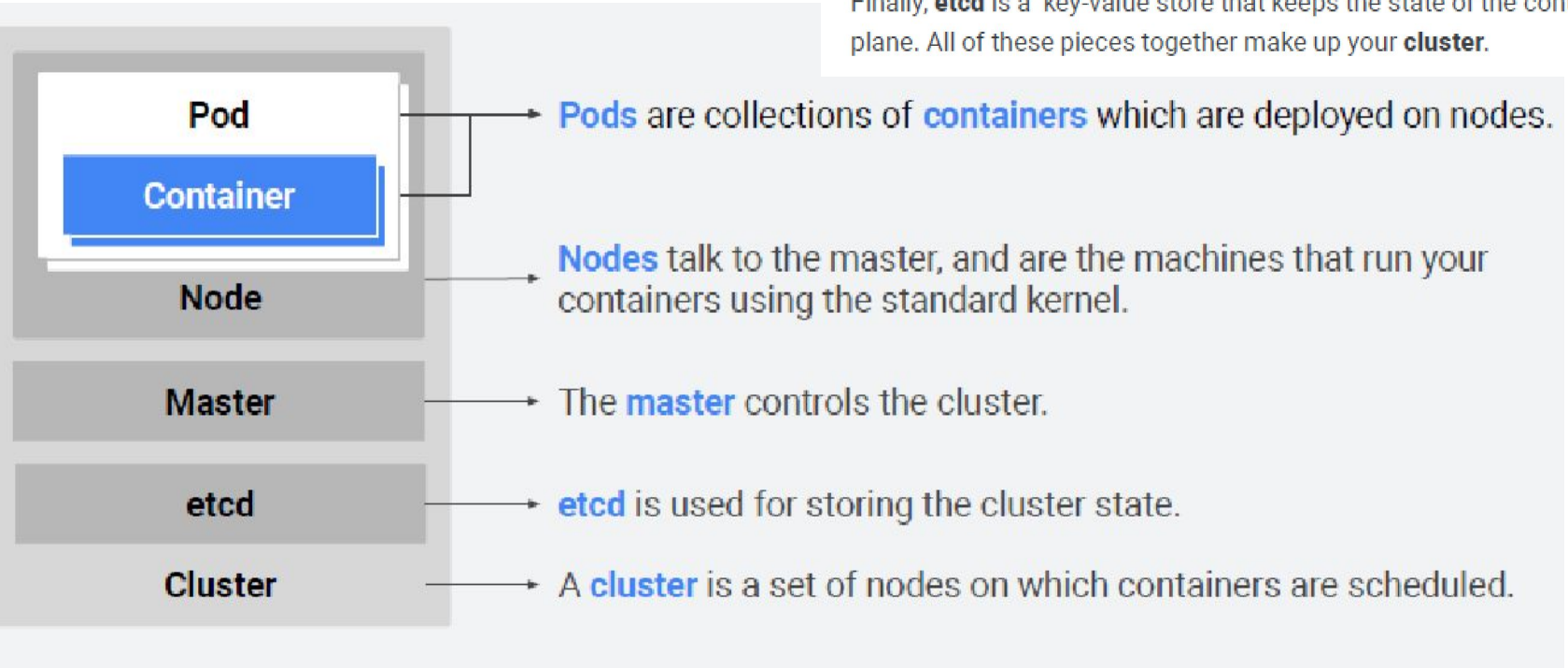
# Kubernetes and Docker in Azure Environment




https://azure.microsoft.com/en-us/topic/kubernetes-vs-docker/#:~:text=A%20fundamental%20difference%20between%20Kubernetes,production%20in%20an%20efficient%20manner

# Kubernetes Architecture

In Kubernetes, a container runs in a **pod**, which in turn runs on a **node**, a virtual or physical machine. The nodes running pods are called **worker nodes**, which contain the container runtime, have their own operating system, and are managed by the **Kubernetes control plane**. Finally, **etcd** is a key-value store that keeps the state of the control plane. All of these pieces together make up your **cluster**.





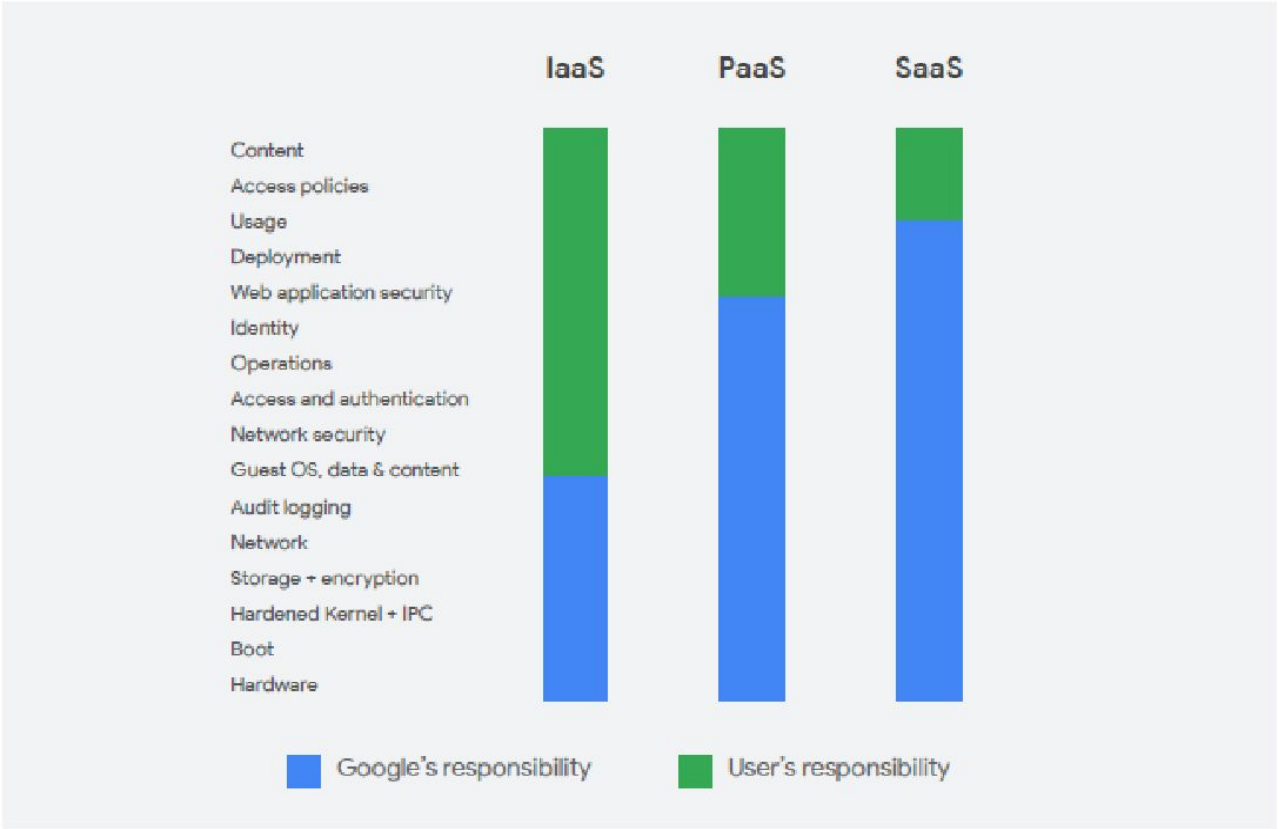

https://services.google.com/fh/files/misc/why_container_security_matters.pdf

https://www.dynatrace.com/news/blog/openshift-vs-kubernetes/#:~:text=Kubernetes%20provides%20a%20basic%20networking,finer%20control%20over%20container%20communication.

# Openshift vs Kubernetes

- Like Kubernetes, OpenShift is an open source Kubernetes-based container platform.
- Developed by Red Hat and can run in a variety of environments — both cloud and on premises (a frequent choice for running Kubernetes on premises)
- Because it's based on Kubernetes, OpenShift provides containerization and orchestration of containerized workloads. Like Kubernetes, it allocates resources efficiently and ensures high availability and fault tolerance
- OpenShift also provides integrated development tools, CI/CD (continuous integration/continuous deployment) pipelines, and built-in support for popular programming languages, frameworks, and databases.
- Red Hat OpenShift helps organizations administer Kubernetes more efficiently. For example, OpenShift simplifies Kubernetes management tools, giving developers everything they need to manage Kubernetes nodes, as well as the underlying control plane.
- OpenShift limitations? Kubernetes supports all cloud and Linux distributions vs. OpenShift supports only Red Hat distributions, such as Red Hat Enterprise Linux (RHEL), CentOS, and Fedora. OpenShift is also comparatively less flexible than open source Kubernetes, making it less customizable to an org's reqmts

# Shared Responsibility Model and Kubernetes PaaS (revisited)




*Shared responsibility on Google Cloud*

# Customer Responsibilities with Kubernetes

- **Nodes** that run workloads
  - Any extra software installed on nodes
  - Config changes made to default
  - Updates to nodes
- **Workloads** themselves
  - Including:
    - Application code
    - Docker files
    - Workload images
    - Data
    - RBAC/IAM policy
    - Containers and Pods being run



https://services.google.com/fh/files/misc/why_container_security_matters.pdf

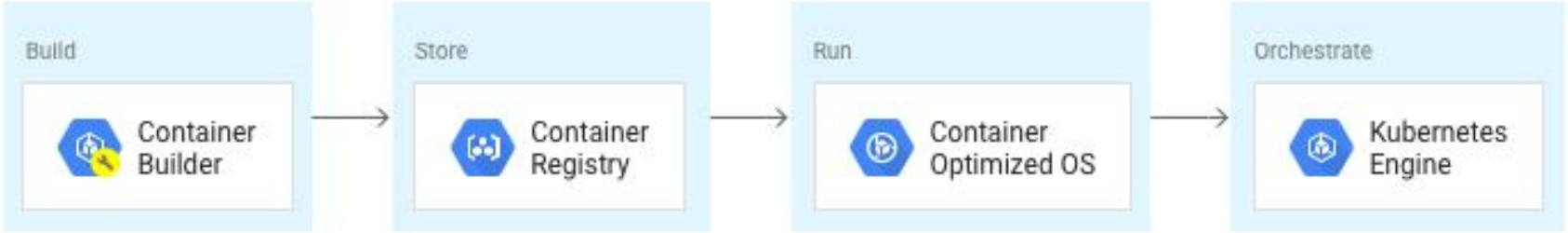
# Cloud Provider Responsibilities with Kubernetes

- Underlying **infrastructure**
  - Hardware, firmware, kernel, OS, storage, network etc.
  - Encryption of data at rest by default, data in transit
  - Data centre protection, network protection
- **Node operating systems**
  - Container Optimized OS (COS) or Ubuntu
  - GKE makes all available patches to these images
- **Kubernetes distribution**
  - GKE provides latest upstream versions of Kubernetes
- **Control Plane** in **GKE**
  - Master VMs, API server and other components on those VMs
  - Etcd database
- Cloud **Platform integrations**
  - E.g. IAM, Cloud Audit Logging, Key Management Services, etc.



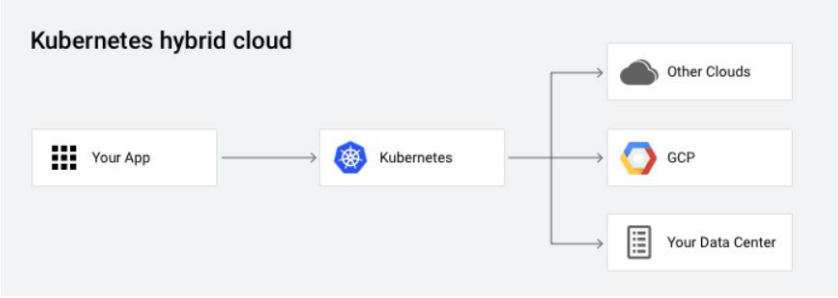
https://services.google.com/fh/files/misc/why_container_security_matters.pdf

# Containers from Development to Production

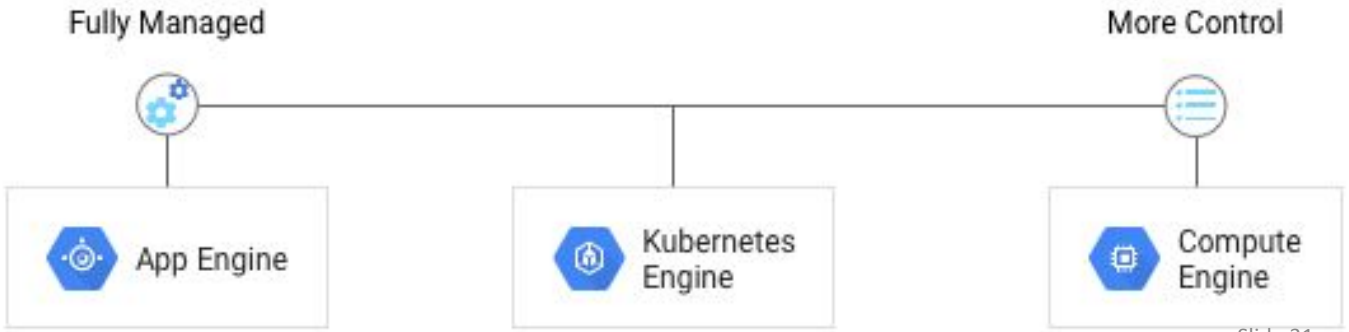



- Tools to use containers from development to production:
  - Cloud Build and Container Registry provide Docker image storage and management on GCP
  - On GCP a Container-Optimized OS provides a lightweight, highly secure operating system that comes with the Docker and Kubernetes runtimes pre-installed.

# Container Run Options




- Cloud Providers typically offer a full spectrum for running containers.
- Fully managed environment e.g. Google Cloud Run to
- Cluster management with Kubernetes Engine to
- Roll-it-yourself infrastructure on e.g. Google Compute Engine



# Container Security Scanning

- **Microsoft** have recently **partnered** with **Qualys** for scanning of Azure Container registries as part of Azure Security Centre

- **Amazon ECR** image scanning helps in identifying software vulnerabilities in your container images. Amazon ECR uses the Common Vulnerabilities and Exposures (CVEs) database from the open source Clair project and provides a list of scan findings.

- **Google Container Analysis** performs vulnerability scans on images in Container Registry and monitors the vulnerability information to keep it up to date. This process comprises two main tasks: scanning and continuous analysis.

# Cloud Workload Security, Q4 2019

“Customer needs in securing workloads are changing. Old-school, on-premises security tooling (security analytics/SA/SIEM), old endpoint detection and response (EDR) solutions, and vanilla password vaulting no longer cut it: Today’s organizations must not only monitor and control the proliferation of cloud workloads but also do it comprehensively across multiple tiers. “

As a result of these trends, cloud workload security customers should look for providers that:

- Offer **solutions for guest operating system native protection.**
- Provide **templatized API-level configuration management to IaaS & PaaS platforms**.
- **Secure container runtimes** and orchestration platforms natively.

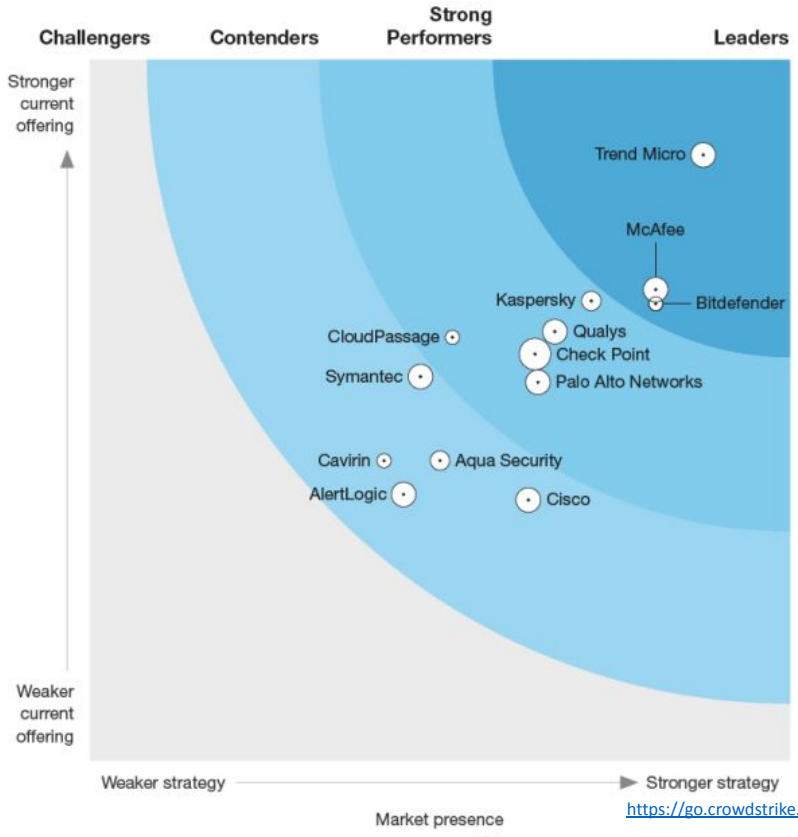


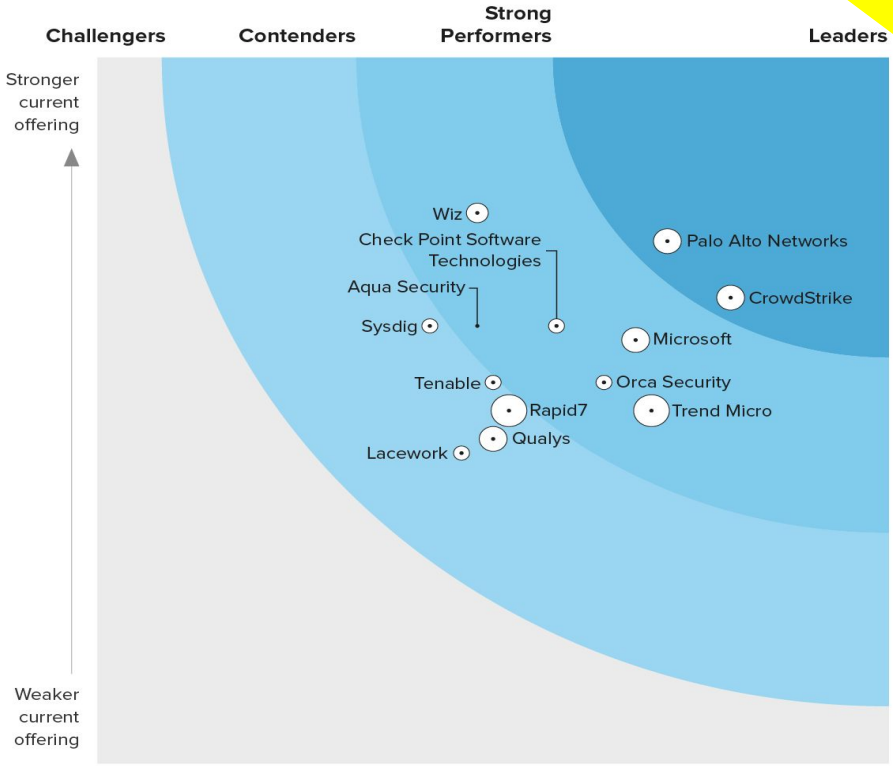

Source: The Forrester Wave™: Cloud Workload Security, Q4 2019

Lucerne University of Applied Sciences and Arts
HOCHSCHULE LUZERN

# Cloud Workload Security - Update Q1/24

This is still the most recent report created by Forrester (as at HS2026)!




https://go.crowdstrike.com/report-the-forrester-wave-cloud-workload-security-q1-2024.html

# Cloud Workload Security Customer Requirements

- Offer **solutions for guest operating system native protection.**
  - Many of the threats in workloads are still traditional changes to configuration files and network intrusions. S&R pros should look for CWS solutions that offer tools-file integrity monitoring, memory integrity monitoring, host-based firewalls, and intrusion detection/prevention and allow for scalable deployment of protection to a large number of workloads without interruption. Leading solutions also start to apply and expose machine learning algorithms and their tuning for admins
- Provide **templatized API-level configuration management** to IaaS and PaaS platforms.
  - You can't control Amazon Web Services (AWS), Azure, or Google Cloud Platform (GCP) using oldschool, on-premises CMDB tools. Instead, you want tight control over instance and storage creation and network connectivity. Best practices, vulnerability, and compliance templates (CIS, CVE, or HIPAA) built into and consistently updated by vendors for managing configurations are key differentiators in this area. ›
- **Secure container runtimes** and **orchestration platforms** natively.
  - With Kubernetes and Docker becoming de facto container environments mainly deployed on cloud platforms, S&R professionals need to be sure that: 1) they scan container images pre-runtime and runtime; 2) there are controls for any configuration drifts at the container level; and 3) they monitor network communications and system calls among containers as well as between containers and the underlying host operating system. Other differentiators include vendor-supplied and constantly updated best practices and compliance templates as well as agentless and agent-based container architectures

Source: The Forrester Wave™: Cloud Workload Security, Q4 2019

# Example Product Positioning: McAfee



# Example Product Positioning: Palo Alto

- Prisma Cloud can scan container images in both public and private repositories on both public and private registries (storing and distributing container images). This includes Docker Hub (most well-known public registry), registries from Amazon, Google, and others or internal private registries set up by organisations.

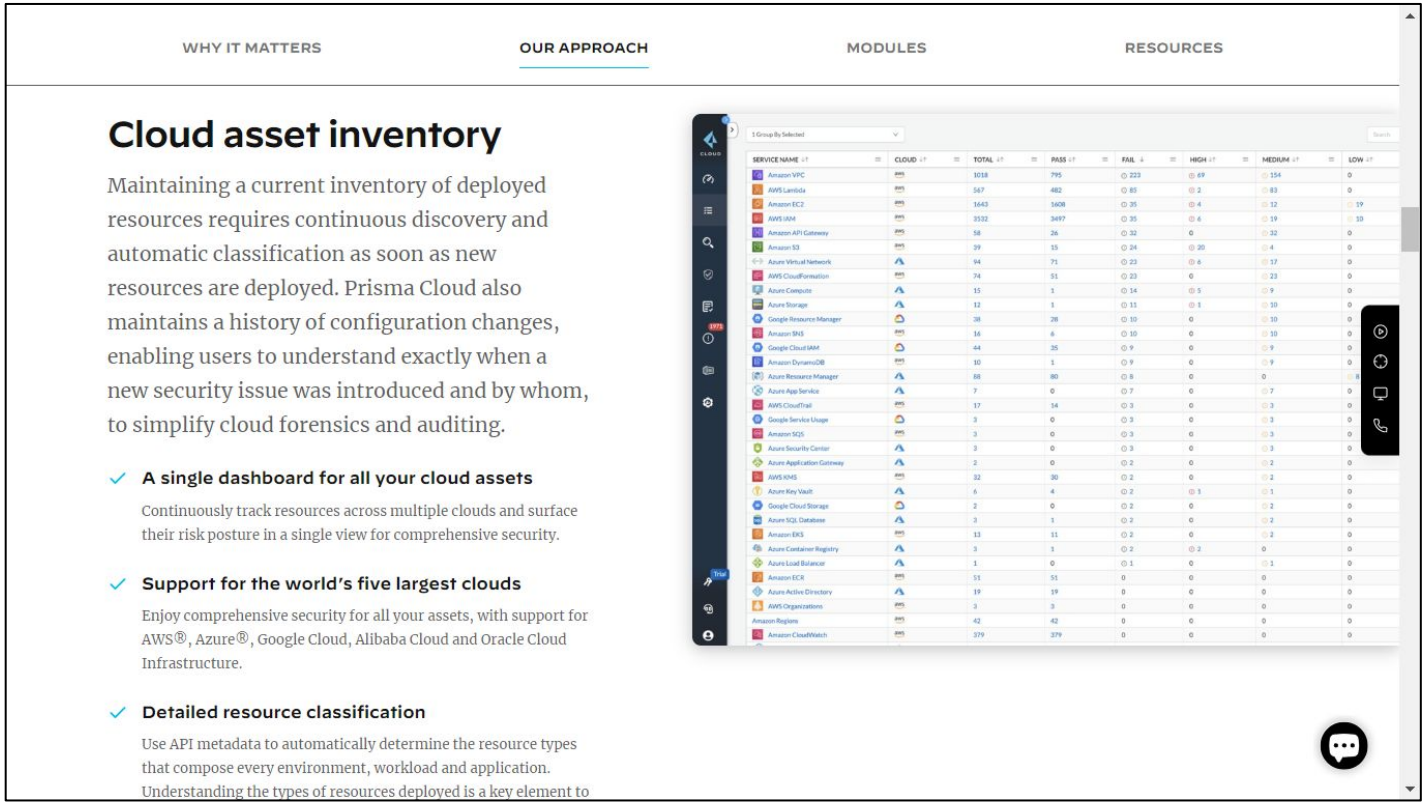
- After repository scanning is configured, Prisma Cloud automatically scans images for vulnerabilities. Periodic scans are run at an interval specified in **Configure > System > Scan** (by default, once every 24 hours).

- Prisma Cloud is a comprehensive **Cloud-Native Application Protection Platform (CNAPP)** that protects infrastructures, workloads, and applications across the entire cloud-native technology stack with industry-leading security and compliance features – throughout the entire development lifecycle in hybrid and multi-cloud environments.

https://www.paloaltonetworks.de/resources/datasheets/prisma-cloud-at-a-glance

# Palo Alto - multi cloud environment

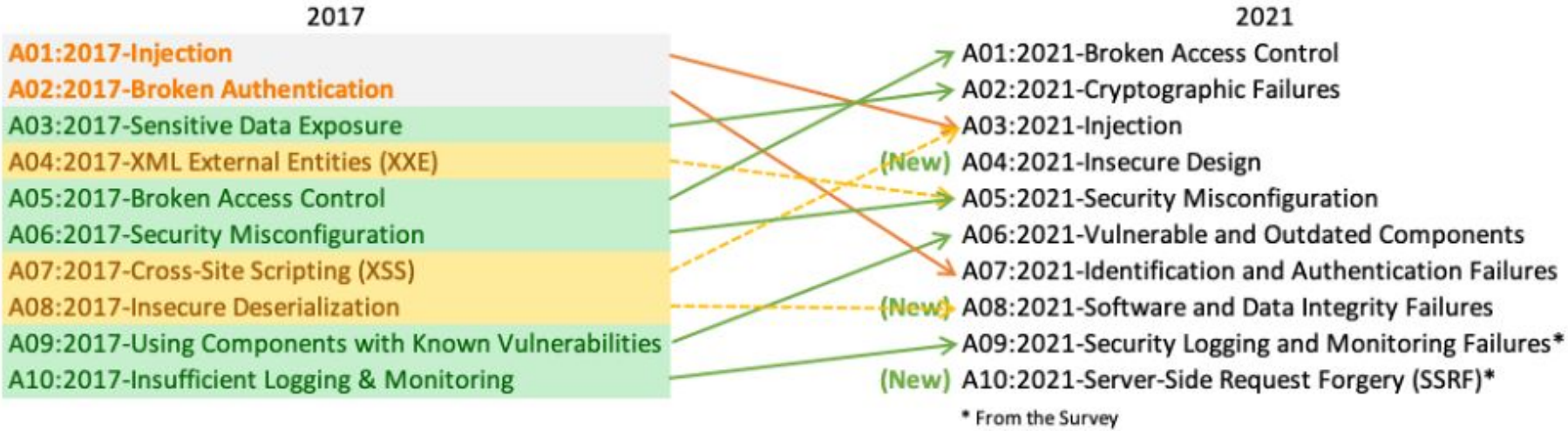
Cloud Security Posture Management




See: https://www.paloaltonetworks.com/prisma/cloud/visibility-compliance-governance

# OWASP Top 10 Security Risks & Vulnerabilities

- **Open Web Application Security Project**
- **The Top 10 OWASP vulnerabilities in 2021 vs 2017 are:**






See: https://cloud.google.com/architecture/owasp-top-ten-mitigation

See: https://owasp.org/Top10/

# OWASP Cloud-Native Application Security Top 10 (interim list, under review)

This is still the most recent status as at HS2026!

- The OWASP Cloud-Native Top 10 list is currently under development (July 2021). As part of our effort to collect feedback, we are presenting an interim list below. Please feel free to contact the project leaders if you have any feedback.
- **CNAS-1: Insecure cloud, container or orchestration configuration**
- **CNAS-2: Injection flaws (app layer, cloud events, cloud services)**
- **CNAS-3: Improper authentication & authorization**
- **CNAS-4: CI/CD pipeline & software supply chain flaws**
- **CNAS-5: Insecure secrets storage**
- **CNAS-6: Over-permissive or insecure network policies**
- **CNAS-7: Using components with known vulnerabilities**
- **CNAS-8: Improper assets management**
- **CNAS-9: Inadequate 'compute' resource quota limits**
- **CNAS-10: Ineffective logging & monitoring (e.g. runtime activity)**



See: https://owasp.org/www-project-cloud-native-application-security-top-10/

# Supply Chain Security with Containers

- Containers can be used to improve overall security posture:
  - Containers **short lived** and **frequently re-deployed** = can constantly be patching
  - Containers are **intentionally immutable** = a modified container is an implicit security alert
  - **Good security defaults** are one line changes = setting security configs is easy
  - With **isolation** technologies, can increase security without adding resources



https://services.google.com/fh/files/misc/why_container_security_matters.pdf

# Quiz 8

- Can you explain the difference between Kubernetes and Docker?
- What is the benefit of scanning containers from a security perspective?
- Where does the name Kubernetes come from?

